

Conception Fonctionnelle

Inventaires_completude_representativite

Boite Eddy

23 août 2026

Résumé

Ce document présente, selon une structure IMRaD, la conception fonctionnelle du pipeline R `Inventaires_completude_representativite.R` (v1.4), dédié à l'analyse de complétude et de représentativité d'inventaires fongiques par site. Les méthodes décrivent les exigences d'entrée, la configuration centralisée, la validation CSV et les indicateurs calculés (TEE, indice de représentativité, complétude). Les résultats formalisent l'organisation thématique des sorties, les livrables globaux et par site, le système de journalisation ainsi que les améliorations successives du script. La discussion précise l'interprétation des indicateurs, les limites opérationnelles et les points de dépannage courants.

Table des matières

1	Introduction	3
1.1	À propos	3
1.2	Objectifs du pipeline	3
1.3	Dépendances et pré-requis	3
2	Méthodes	3
2.1	Structure attendue des données d'entrée	3
2.1.1	Fichier principal	3
2.1.2	Colonnes	3
2.1.3	Règles fonctionnelles	3
2.2	Configuration centralisée	4
2.3	Validation CSV	4
2.4	Indicateurs calculés	4
2.4.1	TEE	4
2.4.2	Indice de représentativité	4
2.4.3	Complétude	5
2.5	Variables d'environnement supportées	5
2.6	Modes d'exécution	5
3	Résultats	5
3.1	Organisation thématique des sorties	5
3.2	Sorties globales (<code>results/ICR/</code>)	6
3.3	Sorties par site (<code>results/ICR/sites/<site_name>/</code>)	6
3.4	Système de logging	6
3.5	Améliorations du script (v1.4)	7
3.5.1	Phase 1 : Fiabilisation	7
3.5.2	Phase 2 : Performance	7
3.5.3	Phase 3 : Graphiques et visualisations	7
3.5.4	Phase 4 : Industrialisation	7

4	Discussion	7
4.1	Journalisation fonctionnelle	7
4.2	Limites et perspectives	7
4.3	Dépannage (FAQ)	7
	Contact et versionnement	8

1 Introduction

1.1 À propos

Ce document formalise les exigences fonctionnelles du script :

`scripts/Inventaires_completude_representativite.R`

(v1.4), destiné à l'analyse de complétude et de représentativité d'inventaires fongiques.

1.2 Objectifs du pipeline

Le pipeline réalise, pour chaque site :

1. la courbe temps-espèces,
2. les ajustements linéaire/hyperbolique (si conditions remplies),
3. l'estimation d'asymptote,
4. le calcul de TEE et de $Ir = 1 - TEE$,
5. les fréquences d'occurrence par espèce,
6. l'occupation spatiale (si `placette`),
7. la CA/AFC (si `vegan`),
8. les métriques de pertinence scientifique,
9. le manifeste final de production.

1.3 Dépendances et pré-requis

Packages requis : `dplyr`, `tidyr`, `ggplot2`, `purrr`, `readr`, `stringr`, `forcats`, `tibble`, `scales`, `gridExtra`.

Packages optionnels : `minpack.lm` (modèle hyperbolique), `vegan` (CA/AFC).

2 Méthodes

2.1 Structure attendue des données d'entrée

2.1.1 Fichier principal

`data/observations.csv` (ou `.txt/.tsv`)

2.1.2 Colonnes

- Obligatoires : `site`, `date`, `visite_id`, `espece`
- Optionnelle : `placette`

2.1.3 Règles fonctionnelles

- Parsing date via `INVENTAIRES_DATE_FORMAT` (défaut : `%Y-%m-%d`).
- Déduplication sur (`site`, `placette`, `date`, `visite_id`, `espece`).
- Visite distincte : `date` + `visite_id` (et `site` en multi-sites).

2.2 Configuration centralisée

Tous les paramètres du pipeline sont définis dans la fonction `get_embedded_config()` et affichés dans l'en-tête du script au démarrage. La configuration supporte les surcharges via variables d'environnement `INVENTAIRES_*` :

- `INVENTAIRES_INPUT_FILE` : chemin du fichier source
- `INVENTAIRES_OUTPUT_DIR` : répertoire de sortie
- `INVENTAIRES_DATE_FORMAT` : format des dates
- `INVENTAIRES_CSV_STRICT` : mode strict (booléen)
- `INVENTAIRES_CSV_ALLOW_EXTRA_COLS` : tolérance colonnes supplémentaires
- `INVENTAIRES_FREQ_BREAKS` : séquence des intervalles de fréquence
- `INVENTAIRES_FREQ_LABELS` : étiquettes des intervalles
- `INVENTAIRES_MAKE_CA` : activer CA/AFC (booléen)
- `INVENTAIRES_MIN_VISITS_FOR_MODEL` : seuil de modélisation
- `INVENTAIRES_WIDTH`, `INVENTAIRES_HEIGHT`, `INVENTAIRES_DPI` : paramètres graphiques

Aucune modification du code source

Les paramètres peuvent être ajustés par variable d'environnement, sans toucher au script.

2.3 Validation CSV

Le contrôle de conformité est exécuté avant traitement.

- `results/ICR/ICR_00_csv_conformite_report.csv` (toujours)
- `results/ICR/ICR_00_csv_conformite_problems.csv` (si anomalies)

Mode strict

En mode strict (`INVENTAIRES_CSV_STRICT=TRUE`), la non-conformité est bloquante.

2.4 Indicateurs calculés

2.4.1 TEE

Proportion d'espèces observées une seule fois.

2.4.2 Indice de représentativité

Formule

$$Ir = 1 - TEE$$

Repères : $Ir < 0.60$ (faible), $0.60 \leq Ir < 0.80$ (moyenne), $Ir \geq 0.80$ (bonne).

2.4.3 Complétude

Formule

$$\text{Complétude} = \frac{S_{obs}}{S_{asymptote}}$$

Repères : < 0.70 (insuffisante), 0.70-0.90 (intermédiaire), >= 0.90 (avancée).

2.5 Variables d'environnement supportées

- INVENTAIRES_INPUT_FILE
- INVENTAIRES_OUTPUT_DIR
- INVENTAIRES_DATE_FORMAT
- INVENTAIRES_CSV_STRICT
- INVENTAIRES_CSV_ALLOW_EXTRA_COLS
- INVENTAIRES_AUTO_RUN

2.6 Modes d'exécution

Le script peut être exécuté en mode standard (recommandé) ou avec des flags internes :

- `DEBUG_MODE = TRUE` : verbosité accrue et affichage détaillé des avertissements.
- `BENCHMARK_MODE = TRUE` : mesure des temps sur les calculs principaux.
- `CLEAN_ENVIRONMENT = TRUE` : nettoyage de l'environnement avant exécution (usage expert).

Le comportement d'auto-lancement est piloté par `INVENTAIRES_AUTO_RUN` (défaut : `TRUE`).

3 Résultats

3.1 Organisation thématique des sorties

Les résultats sont organisés selon une hiérarchie thématique sous `results/ICR/` :

- `00_data_quality/` : rapports d'audit CSV et conformité
- `01_richesse/`, `02_completude_repr/`, `03_frequences/` : analyses par site (réservées)
- `04_spatial/`, `05_temporal/` : analyses spéciales (réservées)
- `06_metrics/` : métriques scientifiques globales
- `comparisons/` : visualisations comparatives multi-sites
- `sites/<site_name>/` : tous les fichiers par site (organisés en un seul répertoire)

3.2 Sorties globales (results/ICR/)

- ICR_donnees_preparees.csv
- ICR_resume_tous_sites.csv
- ICR_metrics_pertinence_tous_sites.csv
- ICR_comparaison_completude_sites.png (dans comparisons/)
- ICR_comparaison_ir_sites.png (dans comparisons/)
- ICR_metrics_pertinence_heatmap_sites.png (dans comparisons/)
- ICR_metrics_pertinence_score_sites.png (dans comparisons/)

3.3 Sorties par site (results/ICR/sites/<site_name>/)

Tous les fichiers CSV et PNG d'un site sont consolidés dans un seul répertoire :

- ICR_01_courbe_temps_especes.csv et graphique PNG
- ICR_02_tee_ir.csv et graphique PNG
- ICR_03_frequence_especes.csv et graphique PNG
- ICR_04_resume_site.csv
- ICR_07_metrics_pertinence.csv et graphique PNG
- ICR_05_modeles.csv (si conditions remplies)
- ICR_06_occupation_spatiale.csv (si placette)
- ICR_05_temporel_vs_spatial.png (conditionnelle)
- ICR_06_CA_placettes_especes.png (si vegan et placette)

3.4 Système de logging

Chaque exécution du script génère un fichier log horodaté dans le répertoire logs/ du projet :

- **Format du nom** : ICR_YYYYMMDD_HHMM.log
- **Format des entrées** : [YYYY-MM-DD HH:MM:SS] [LEVEL] message
- **Niveaux** : INFO, DEBUG, WARN

Les logs enregistrent :

1. l'en-tête du script (nom, version, timestamp, répertoire de travail, version R) ;
2. chaque étape du pipeline (résolution fichier, audit CSV, analyses par site) ;
3. les paramètres de configuration actifs au démarrage ;
4. le manifeste final des livrables avec statuts (OK, MANQUANT, OPTIONNEL_NON_GENERE).

Les avertissements sont également affichés en console et dans les logs en niveau WARN.

3.5 Améliorations du script (v1.4)

3.5.1 Phase 1 : Fiabilisation

Gestion robuste des formats de fichiers hétérogènes, déduplication intelligente des observations, traitement des dates flexibles, gestion des erreurs CSV granulaire.

3.5.2 Phase 2 : Performance

Optimisation du modèle hyperbolique via `minpack.lm::nlsLM`, calculs vectorisés par site, graphiques pré-calculés en mémoire avant export PNG.

3.5.3 Phase 3 : Graphiques et visualisations

Cartes de chaleur multi-sites, tableaux de bord par site (7 graphiques), comparaisons visuelles de complétude, analyses de CA/AFC si `vegan` disponible.

3.5.4 Phase 4 : Industrialisation

1. **Configuration centralisée** : fonction `get_embedded_config()` avec 14+ paramètres, surchargeable via variables d'environnement
2. **Logging systématisé** : fichiers logs horodatés avec métadonnées d'exécution, manifeste final des livrables
3. **Organisation thématique** : hiérarchie logique `/sites/`, `/comparisons/`, répertoires réservés pour extensions futures
4. **Métriques scientifiques** : pertinence des espèces, scores de complétude détaillés, dashboards analytiques

4 Discussion

4.1 Journalisation fonctionnelle

La sortie est journalisée en console (niveaux info/debug/warning), puis un manifeste final indique l'état des livrables : OK, MANQUANT, OPTIONNEL_NON_GENERE.

Le script applique enfin une politique stricte de nettoyage du fichier parasite `Rplots.pdf` lorsqu'il est généré pendant l'exécution.

4.2 Limites et perspectives

La disponibilité de certaines sorties dépend de la richesse du jeu de données fourni : sans la colonne `placette`, l'analyse spatiale n'est pas produite ; sans le package `minpack.lm`, l'asymptote hyperbolique n'est pas estimée ; sans le package `vegan`, la CA/AFC n'est pas générée ; en deçà du seuil `INVENTAIRES_MIN_VISITS_FOR_MODEL`, la modélisation est ignorée. En mode strict, un CSV non conforme bloque le pipeline.

4.3 Dépannage (FAQ)

Le script ne trouve pas le fichier d'entrée. Vérifier qu'un fichier `.csv`, `.txt` ou `.tsv` est présent dans `data/` et que son nom correspond à `INVENTAIRES_INPUT_FILE`.

Colonnes manquantes à l'exécution. Vérifier que les colonnes obligatoires `site`, `date`, `visite_id`, `espece` sont présentes avec les noms exacts attendus.

Le pipeline s'arrête en mode strict. Consulter `results/ICR/ICR_00_csv_conformite_problems.csv` pour identifier les anomalies, ou désactiver temporairement `INVENTAIRES_CSV_STRICT`.

Pas d'occupation spatiale ni de CA/AFC. Vérifier la présence de la colonne `placette` et, pour la CA/AFC, l'installation du package `vegan`.

Pas d'asymptote hyperbolique. Vérifier l'installation du package `minpack.lm` et que le nombre de visites du site atteint `INVENTAIRES_MIN_VISITS_FOR_MODEL`.

Contact et versionnement

Auteur	Eddy Boite
Dépôt GitHub	<code>eddyboite-nat/myco_apps_releases</code>
Branche	<code>main</code>
Version script	1.1
Date de ce document	9 juillet 2026

Pour toute évolution du script, ouvrir une issue dans le dépôt GitHub ou documenter les changements dans le journal `CHANGELOG.md` du projet.